

FINAL YEAR PROJECT 2

Project Continuity and Technical Handover Report

Unsupervised Candlestick Visual-Regime Discovery in Google Colab

Report purpose	Restore complete working context in a future ChatGPT session
Status date	29 July 2026
Current stage	Colab setup prepared; training is about to be restarted
Dataset variant	autoscaled
Primary code archive	MyDrive/final_year_project_2.zip

Read this report first in the next session. It is designed as a complete memory-restoration document.

1. Immediate Memory Restoration

One-paragraph project state: The project uses candlestick-chart images to discover visual market regimes without labels. The correct code package is stored in MyDrive/final_year_project_2.zip. The active dataset is MyDrive/final_year_project_2_datasets/autoscaled, with manifests in manifests/ and images in images/. The CSV image paths were created on Windows and must be converted to absolute Colab Drive paths. The latest check found 23,568 training rows, of which 752 point to missing image files, and 2,560 validation rows with zero missing files. The agreed working solution is to create temporary corrected manifests under /content/regime_discovery_dataset/manifests, remove only the 752 missing training rows from that temporary copy, update the extracted config.json, run the pipeline on a Colab GPU, and finally copy outputs from /content/candlestick_regime_discovery_output to MyDrive/final_year_project_2_outputs/autoscaled_regime_discovery.

Current truth table

Item	Confirmed state	Confidence / note
Correct archive	/content/drive/MyDrive/final_year_project_2.zip	Confirmed by uploaded ZIP inspection
Code package folder	final_year_project_2/candlestick_regime_discovery_package	Confirmed inside ZIP
Dataset root	/content/drive/MyDrive/final_year_project_2_datasets/autoscaled	Confirmed by user
Training manifest	manifests/train_samples.csv	Confirmed
Validation manifest	manifests/validation_samples.csv	Confirmed
Image folder	images/	Confirmed
Training rows	23,568 before cleaning	Latest Colab output supplied by user
Missing training images	752	Latest result; supersedes an earlier screenshot showing a different count
Validation rows	2,560	Latest Colab output supplied by user
Missing validation images	0	Latest Colab output supplied by user
Training run	Not yet confirmed complete	User said they were about to

Item	Confirmed state	Confidence / note
		restart training

Non-negotiable next-session rule

Do not claim that training completed or that final clusters were obtained unless output files are uploaded or the user confirms completion.

2. Project Purpose and Analytical Pipeline

The package performs an unsupervised visual-regime discovery stage using candlestick images. It does not use predefined regime labels. The output clusters are visual groupings and must not automatically be interpreted as economic regimes without visual and numerical validation.

1. Read the training and validation manifests.
2. Load each candlestick image and preprocess it using the ResNet-18 weight transforms.
3. Use a frozen ImageNet-pretrained ResNet-18 with its classification layer replaced by an identity layer to produce 512-dimensional embeddings.
4. Fit PCA only on training embeddings and use the fitted PCA to transform validation embeddings.
5. Evaluate candidate K-Means cluster counts $K = 4, 6, 8, 10$, and 12 by default.
6. Calculate silhouette, Davies-Bouldin, Calinski-Harabasz, inertia, stability, minimum-cluster proportion, and cluster entropy/balance metrics.
7. Recommend K using a weighted normalized score, unless `selected_k` is manually configured.
8. Fit final K-Means on training PCA features only and predict validation cluster assignments.
9. Save models, embeddings, assignments, plots, representative images, random samples, and montages.

Automatic K recommendation weights

Metric component	Direction	Weight
Silhouette	Higher is better	0.25
Stability (mean ARI)	Higher is better	0.25
Davies-Bouldin	Lower is better	0.15
Minimum cluster proportion	Higher is better	0.15
Calinski-Harabasz	Higher is better	0.10
Normalized cluster entropy	Higher is better	0.10

Interpretation warning: Clusters are visual groups. Labels such as bullish, bearish, volatile, or sideways require inspection of `montage_nearest.png` and `montage_random.png` and ideally confirmation using market statistics.

3. Source Files Already Reviewed

Source	What it established	Status
--------	---------------------	--------

final_year_project_2.zip	Correct code package, README, config, requirements, batch helpers, and main Python pipeline	Reviewed
run_unsupervised_regime_discovery.py	Full implementation of embedding extraction, PCA, K selection, final clustering, reporting, and montages	Reviewed
config.json	Original Windows dataset/output paths and default hyperparameters	Reviewed
README.md	Intended package usage, leakage rule, expected outputs, and interpretation caution	Reviewed
requirements.txt	Python dependency minimum versions	Reviewed
Colab screenshot PDF	Prior notebook state and evidence of manifest conversion/missing image diagnostics	Reviewed

4. Confirmed Drive and Package Structure

Google Drive

```

MyDrive/
├── final_year_project_2.zip
├── final_year_project_2_datasets/
│   ├── autoscaled/
│   │   ├── manifests/
│   │   │   ├── train_samples.csv
│   │   │   ├── validation_samples.csv
│   │   │   ├── all_samples.csv
│   │   │   ├── duplicate_images.json
│   │   │   └── dataset_summary.json
│   │   ├── images/
│   │   │   └── sample_<start>_<stop>.jpg
│   │   └── skipped_samples.txt
└── final_year_project_2_outputs/      # to be created/used for permanent results

```

Correct ZIP contents

```

final_year_project_2/
├── candlestick_regime_discovery_package/
│   ├── config.json
│   ├── README.md
│   ├── requirements.txt
│   ├── run_unsupervised_regime_discovery.py
│   ├── install_requirements.bat
│   └── run_pipeline.bat

```

Temporary Colab working paths

```
/content/final_year_project_2_code/      # extracted ZIP
/content/regime_discovery_dataset/manifests/  # corrected working manifests
/content/candlestick_regime_discovery_output/  # temporary pipeline output
```

5. Absolute Windows Path Problem and Correct Fix

The manifest column `exported_image` stores absolute Windows paths. A typical original value is:

```
C:
\Users\m0zim\OneDrive\Desktop\final_year_project_2_datasets\btc_1h_unsupervised_candlestick_dataset
\images\sample_0_29.jpg
```

Colab cannot open that path. The conversion must normalize backslashes, retain only the filename, and rebuild an absolute Google Drive path:

```
/content/drive/MyDrive/final_year_project_2_datasets/autoscaled/images/sample_0_29.jpg
```

The original manifests in Drive should remain unchanged. Corrected copies are created in Colab temporary storage.

6. Missing-Image Finding and Agreed Handling

The latest supplied conversion output was:

```
train_samples.csv
Rows: 23,568
Missing images: 752

validation_samples.csv
Rows: 2,560
Missing images: 0
```

Examples of missing training files include `sample_9_38.jpg`, `sample_92_121.jpg` through `sample_99_128.jpg`, and `sample_929_958.jpg`. The path conversion is correct; these files are absent from the specified `autoscaled/images` folder.

Agreed operational decision: Remove those 752 rows only from the temporary working training manifest, save a `missing_train_images.csv` audit report, and keep the original Drive manifest untouched. The expected clean training size is 22,816 rows.

This is a practical run-enabling decision, not yet a fully justified research-methodology decision. The final dissertation methodology must explain why images are missing and assess whether the exclusions could introduce systematic bias.

7. Exact Fresh-Colab Notebook Procedure

Use one code cell per block, in this exact order. A future assistant should preserve the paths unless the user reports a different Drive structure.

Cell 1 - Mount Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

Cell 2 - Define paths and verify files

```
from pathlib import Path

CODE_ZIP = Path('/content/drive/MyDrive/final_year_project_2.zip')
DATASET_DIR = Path('/content/drive/MyDrive/final_year_project_2_datasets/autoscaled')
MANIFEST_DIR = DATASET_DIR / 'manifests'
IMAGES_DIR = DATASET_DIR / 'images'

checks = {
    'ZIP': CODE_ZIP.is_file(),
    'dataset': DATASET_DIR.is_dir(),
    'manifests': MANIFEST_DIR.is_dir(),
    'images': IMAGES_DIR.is_dir(),
    'train CSV': (MANIFEST_DIR / 'train_samples.csv').is_file(),
    'validation CSV': (MANIFEST_DIR / 'validation_samples.csv').is_file(),
}
for name, ok in checks.items():
    print(f'{name}: {ok}')

if not all(checks.values()):
    raise FileNotFoundError('One or more required Drive paths are missing.')
```

Cell 3 - Extract the correct ZIP

```
import shutil
import zipfile
from pathlib import Path

EXTRACT_ROOT = Path('/content/final_year_project_2_code')
if EXTRACT_ROOT.exists():
    shutil.rmtree(EXTRACT_ROOT)
EXTRACT_ROOT.mkdir(parents=True)

with zipfile.ZipFile(CODE_ZIP, 'r') as archive:
    archive.extractall(EXTRACT_ROOT)

matches = list(EXTRACT_ROOT.rglob('candlestick_regime_discovery_package'))
if len(matches) != 1:
    raise RuntimeError(f'Expected one package folder, found {len(matches)}')
PACKAGE_DIR = matches[0]
print('PACKAGE_DIR =', PACKAGE_DIR)
for path in sorted(PACKAGE_DIR.iterdir()):
    print(' ', path.name)
```

Cell 4 - Install requirements and verify GPU

```
%cd $PACKAGE_DIR
!python -m pip install -q -r requirements.txt

import torch
import torchvision
print('PyTorch:', torch.__version__)
print('Torchvision:', torchvision.__version__)
print('CUDA available:', torch.cuda.is_available())
if torch.cuda.is_available():
    print('GPU:', torch.cuda.get_device_name(0))
```

Cell 5 - Inspect source manifests

```
import pandas as pd

train_source = MANIFEST_DIR / 'train_samples.csv'
validation_source = MANIFEST_DIR / 'validation_samples.csv'
train_df = pd.read_csv(train_source)
validation_df = pd.read_csv(validation_source)

print('Training rows:', len(train_df))
print('Validation rows:', len(validation_df))
print('Columns:', train_df.columns.tolist())
print('Original example paths:')
print(train_df['exported_image'].head(3).tolist())
```

Cell 6 - Convert Windows paths into absolute Colab/Drive paths

```

from pathlib import Path
import pandas as pd

WORK_DATASET = Path('/content/regime_discovery_dataset')
WORK_MANIFESTS = WORK_DATASET / 'manifests'
WORK_MANIFESTS.mkdir(parents=True, exist_ok=True)

def convert_manifest(source_path: Path, destination_path: Path) -> pd.DataFrame:
    frame = pd.read_csv(source_path)
    required = {'sample_name', 'start_index', 'stop_index', 'exported_image'}
    missing_columns = required - set(frame.columns)
    if missing_columns:
        raise ValueError(f'{source_path.name} is missing {sorted(missing_columns)}')

    def convert_path(value: str) -> str:
        normalized = str(value).replace('\\', '/')
        filename = normalized.rsplit('/', 1)[-1]
        return str(IMAGE_DIRS / filename)

    frame['exported_image'] = frame['exported_image'].map(convert_path)
    frame.to_csv(destination_path, index=False)

    missing = [p for p in frame['exported_image'] if not Path(p).is_file()]
    print(f'\n{source_path.name}')
    print(f'Rows: {len(frame):,}')
    print(f'Missing images: {len(missing):,}')
    print(f'First converted path:', frame.iloc[0]['exported_image'])
    for path in missing[:10]:
        print(' ', path)
    return frame

train_ready = convert_manifest(train_source, WORK_MANIFESTS / 'train_samples.csv')
validation_ready = convert_manifest(validation_source, WORK_MANIFESTS / 'validation_samples.csv')

```

Cell 7 - Remove missing rows from the temporary training manifest only

```

from pathlib import Path
import pandas as pd

working_train_path = WORK_MANIFESTS / 'train_samples.csv'
working_validation_path = WORK_MANIFESTS / 'validation_samples.csv'
train_working = pd.read_csv(working_train_path)
validation_working = pd.read_csv(working_validation_path)

exists_mask = train_working['exported_image'].map(lambda p: Path(str(p)).is_file())
missing_train_rows = train_working.loc[~exists_mask].copy()
clean_train = train_working.loc[exists_mask].copy()

missing_report = WORK_MANIFESTS / 'missing_train_images.csv'
missing_train_rows.to_csv(missing_report, index=False)
clean_train.to_csv(working_train_path, index=False)

print('Original training rows:', len(train_working))
print('Removed missing rows:', len(missing_train_rows))
print('Clean training rows:', len(clean_train))
print('Validation rows:', len(validation_working))
print('Missing-row report:', missing_report)

```

Cell 8 - Mandatory zero-missing verification

```

from pathlib import Path
import pandas as pd

for manifest_path in [
    WORK_MANIFESTS / 'train_samples.csv',
    WORK_MANIFESTS / 'validation_samples.csv',
]:
    frame = pd.read_csv(manifest_path)
    missing_count = sum(not Path(str(p)).is_file() for p in frame['exported_image'])
    print(manifest_path.name, '| rows:', len(frame), '| missing:', missing_count)
    if missing_count != 0:
        raise RuntimeError(f'{manifest_path.name} still contains missing files')

```

Cell 9 - Update the extracted config.json

```
import json
from pathlib import Path

OUTPUT_DIR = Path('/content/candlestick_regime_discovery_output')
config_path = PACKAGE_DIR / 'config.json'

with config_path.open('r', encoding='utf-8') as file:
    config = json.load(file)

config['dataset_dir'] = str(WORK_DATASET)
config['output_dir'] = str(OUTPUT_DIR)
config['train_manifest'] = 'manifests/train_samples.csv'
config['validation_manifest'] = 'manifests/validation_samples.csv'
config['image_column'] = 'exported_image'
config['device'] = 'auto'
config['batch_size'] = 32
config['num_workers'] = 0
config['cpu_threads'] = 4
config['pause_on_exit'] = False
config['reuse_saved_embeddings'] = True
config['overwrite_output'] = False

with config_path.open('w', encoding='utf-8') as file:
    json.dump(config, file, indent=2)

print(json.dumps(config, indent=2))
```

Cell 10 - Verify configured paths

```
configured_train = Path(config['dataset_dir']) / config['train_manifest']
configured_validation = Path(config['dataset_dir']) / config['validation_manifest']
print('Train manifest:', configured_train, configured_train.is_file())
print('Validation manifest:', configured_validation, configured_validation.is_file())
assert configured_train.is_file() and configured_validation.is_file()
```

Cell 11 - Run the pipeline

```
%cd $PACKAGE_DIR
!python -u run_unsupervised_regime_discovery.py
```

Cell 12 - Permanently save results to Drive

```
import shutil
from pathlib import Path

DRIVE_OUTPUT =
Path('/content/drive/MyDrive/final_year_project_2_outputs/autoscaled_regime_discovery')
if not OUTPUT_DIR.is_dir():
    raise FileNotFoundError(f'Output directory not created: {OUTPUT_DIR}')
DRIVE_OUTPUT.parent.mkdir(parents=True, exist_ok=True)
if DRIVE_OUTPUT.exists():
    shutil.rmtree(DRIVE_OUTPUT)
shutil.copytree(OUTPUT_DIR, DRIVE_OUTPUT)
print('Saved permanently to:', DRIVE_OUTPUT)
```

8. Configuration Values for the Colab Run

Key	Colab value	Reason
dataset_dir	/content/ regime_discovery_dataset	Points to corrected working manifests
output_dir	/content/ candlestick_regime_discovery_ output	Fast temporary local storage
train_manifest	manifests/train_samples.csv	Cleaned temporary training

		CSV
validation_manifest	manifests/ validation_samples.csv	Corrected validation CSV
image_column	exported_image	Matches manifest schema
device	auto	Uses CUDA when available
batch_size	32	Conservative Colab starting value
num_workers	0	Avoids common Colab DataLoader worker issues
pause_on_exit	false	Prevents notebook cell from waiting for input
reuse_saved_embeddings	true	Allows reuse if compatible outputs are restored
overwrite_output	false	Avoids deleting reusable results by default

9. Expected Outputs and Their Meaning

```

candlestick_regime_discovery_output/
├── embeddings/
│   ├── train_embeddings.npy
│   ├── validation_embeddings.npy
│   ├── train_metadata.csv
│   ├── validation_metadata.csv
│   ├── train_pca_features.npy
│   └── validation_pca_features.npy
├── models/
│   ├── pca.joblib
│   └── kmeans.joblib
├── reports/
│   ├── candidate_k_metrics.csv
│   ├── candidate_k_metrics.json
│   ├── candidate_k_recommendation_scores.csv
│   ├── selected_k.json
│   ├── train_cluster_labels.csv
│   ├── validation_cluster_labels.csv
│   ├── cluster_summary.csv
│   └── run_summary.json
├── plots/
│   └── k_selection_*.png
└── clusters/
    └── cluster_XX/
        ├── representative.jpg
        ├── nearest_members.csv
        ├── random_members.csv
        ├── nearest_*.jpg
        ├── random_*.jpg
        ├── montage_nearest.png
        └── montage_random.png

```

Permanent Drive destination

```
/content/drive/MyDrive/final_year_project_2_outputs/autoscaled_regime_discovery
```

Persistence warning: Anything under /content can be lost when the Colab runtime ends. Always copy completed outputs to MyDrive.

10. Troubleshooting and Recovery Rules

Symptom	Action
CUDA available: False	Use Runtime > Change runtime type > T4 GPU, then restart and rerun cells.
Image does not exist	Rerun conversion and zero-missing verification. Do not run pipeline with missing paths.
GPU out of memory	Reduce batch_size from 32 to 16 or 8.
DataLoader freezes/errors	Keep num_workers at 0.
Output directory not empty error	Keep reuse_saved_embeddings=true, or deliberately set overwrite_output=true for a fresh run. Note that overwrite deletes the old output first.
Runtime disconnects before saving	Temporary /content data may be lost. Rerun from Drive or restore any separately backed-up embeddings.
First run pauses at the end	Ensure pause_on_exit=false in the extracted config.json.
ResNet weights download fails	Confirm internet access in the Colab runtime and rerun.

11. Methodology Material Already Supported

The following statements can already be used in a future research methodology, subject to alignment with the full dataset-generation files and research objectives:

- Research design: quantitative, computational, exploratory, and unsupervised machine-learning design.
- Unit of analysis: a fixed candlestick-image window represented by one manifest row and one image file.
- Data partitioning: separate training and validation manifests; PCA and K-Means are fitted on training data only.
- Representation learning: transfer learning through a frozen ImageNet-pretrained ResNet-18 used as a feature extractor, not fine-tuned.
- Dimensionality reduction: PCA fitted on the training embeddings and then applied to validation embeddings.
- Clustering: K-Means with multiple initializations and a fixed random seed.
- Model-selection evidence: internal validity, separation, compactness, balance, and repeated-run stability metrics.

- Qualitative validation: representative and random cluster montages must be inspected before semantic interpretation.
- Reproducibility controls: seeded random number generators, saved models, saved embeddings, configuration file, metadata compatibility checks, and machine-readable reports.

12. Additional Files and Facts Required Next Time

The next session should request the following before producing the final dissertation methodology. Each item is needed to avoid inventing unsupported details.

Required item	Why it is needed	Priority
Dataset-generation script(s), especially <code>prepare_unsupervised_candles_tick_dataset.py</code>	Establish source data, candle-window construction, chart rendering, indicators, scaling, image dimensions, overlap, exclusions, and chronological split logic	Critical
<code>dataset_summary.json</code>	Confirm counts, date coverage, interval, skipped samples, and dataset-generation parameters	Critical
<code>duplicate_images.json</code>	Explain duplicate detection and leakage controls	Critical
<code>skipped_samples.txt</code>	Identify why samples were omitted and whether omission may bias the dataset	Critical
Sample rows or full schemas from <code>train_samples.csv</code> , <code>validation_samples.csv</code> , and <code>all_samples.csv</code>	Document all variables, ordering, timestamps, split fields, and index semantics	Critical
Final output folder after the Colab run	Report actual selected K, PCA variance, cluster sizes, metrics, plots, and model artifacts	Critical
Research title, aim, objectives, research questions, and hypotheses if any	Align every methodological choice with the study purpose	Critical
Institutional methodology chapter template, marking rubric, word limit, and citation style	Match required academic structure and assessment criteria	High
Original market-data source and retrieval script or documentation	Document provenance, exchange/provider, symbol, timeframe, fields, period, and	High

	data quality	
Computer/Colab runtime details from the completed run	Report software versions, GPU model, runtime, and reproducibility environment	High
Ethics/data-governance requirements	Address licensing, privacy, market-data terms, and responsible interpretation	Medium
Supervisor decisions on handling the 752 missing training images	Provide defensible exclusion rationale and sensitivity analysis plan	Critical

13. Questions the Next Assistant Must Ask

10. Did the Colab pipeline complete successfully? Upload or describe run_summary.json and selected_k.json.
11. Were the 752 missing-image rows removed exactly as described, and was missing_train_images.csv retained?
12. What is the formal research title, aim, objectives, and set of research questions?
13. What raw market dataset was used: asset, exchange/provider, interval, date range, and fields?
14. How were candlestick images generated: window length, stride, scaling rule, dimensions, colors, axes, indicators, and normalization?
15. How was the train/validation boundary chosen, and is it chronological?
16. What do dataset_summary.json, duplicate_images.json, and skipped_samples.txt report?
17. What methodology format and citation style does the university require?
18. Does the user want a methodology chapter only, a complete dissertation report, or both?

14. Instructions to the Next ChatGPT Session

Role instruction: Treat this document as the authoritative continuity record, but verify all current run results against newly uploaded output files. Do not silently assume that a planned step was completed.

- Start by summarizing the current state back to the user in five to eight sentences.
- Ask for the critical files listed in Section 12, preferably as one ZIP plus the completed output folder.
- Inspect files before drafting; preserve the project terminology and do not replace it with generic methodology language.
- Clearly label source-supported facts, inferences, recommendations, and unresolved questions.
- When the training output is available, calculate and describe actual cluster-selection results rather than discussing only defaults.
- Do not call the process neural-network training in a strict sense: ResNet-18 is frozen and used for inference/feature extraction; PCA and K-Means are fitted.
- Do not label clusters as market regimes until montages and numerical market behaviour support those labels.

- Address the exclusion of missing images transparently and consider a sensitivity/bias assessment.

15. Completion Checklist

Task	Status at report creation
Drive mounted and all paths verified	Pending each new runtime
ZIP extracted	Pending each new runtime
GPU enabled	Pending each new runtime
Windows paths converted	Procedure ready
752 missing training rows removed from working copy	Procedure ready; completion not confirmed
Both working manifests show zero missing images	Must be confirmed before run
config.json updated	Procedure ready; completion not confirmed
Pipeline run completed	Not confirmed
Outputs copied permanently to Drive	Not confirmed
Cluster montages inspected	Not started/confirmed
Full research methodology drafted from all files	Future task

Appendix A. Original Default Configuration

```
{
  "dataset_dir": "C:\\Users\\m0zim\\OneDrive\\Desktop\\final_year_project_2_datasets\\
\\btc_1h_unsupervised_candlestick_dataset",
  "output_dir": "C:\\Users\\m0zim\\OneDrive\\Desktop\\final_year_project_2_outputs\\
\\candlestick_regime_discovery",
  "train_manifest": "manifests/train_samples.csv",
  "validation_manifest": "manifests/validation_samples.csv",
  "image_column": "exported_image",
  "batch_size": 16,
  "num_workers": 0,
  "cpu_threads": 4,
  "image_size": 224,
  "device": "auto",
  "embedding_model": "resnet18",
  "use_pretrained_weights": true,
  "pca_components": 50,
  "candidate_k": [4, 6, 8, 10, 12],
  "selected_k": null,
  "kmeans_n_init": 20,
  "stability_repeats": 5,
  "stability_subsample": 4000,
  "silhouette_subsample": 5000,
  "random_seed": 42,
  "nearest_images_per_cluster": 16,
  "random_images_per_cluster": 16,
  "montage_columns": 4,
  "overwrite_output": false,
  "reuse_saved_embeddings": true,
  "pause_on_exit": true
}
```

Appendix B. Evidence and Uncertainty Notes

- The correct ZIP was inspected directly and contains the package listed in Section 4.
- The main script reads config.json from its own directory and is launched without command-line path arguments.
- An earlier screenshot in the conversation showed a larger missing-image count, while the latest fresh-run text shows 752. This report uses the latest value and flags the discrepancy rather than trying to reconcile it without the actual files.
- The report records planned notebook actions separately from completed actions. At creation time, completion of the new training run was not confirmed.
- No actual clustering output metrics or selected K were available at report creation time.

END OF CONTINUITY REPORT